

Monitoreo de flujo continuo AS7265x

Matias vasquez

March 23, 2026

Abstract

El presente trabajo describe el desarrollo de un sistema de espectrofotometría de flujo continuo de bajo costo para el monitoreo de la calidad del agua en tiempo real. Utilizando un sensor multiespectral AS7265x de 18 canales (UV, Visible, Infrarrojo) controlado por un microcontrolador ESP32, se construyó una celda de flujo con materiales de ferretería (PVC). Los datos capturados vía UART son procesados mediante un script en Python que calcula un "Índice de Alteración" basado en el error absoluto medio respecto a una muestra de referencia. Se implementó un algoritmo de Seguimiento Dinámico de Línea Base para mitigar la deriva térmica de los LEDs y el ruido por microburbujas. Las pruebas de inyección de detergentes comerciales demostraron que el sistema es capaz de detectar anomalías instantáneas, validando su viabilidad como alternativa económica a los equipos industriales tradicionales.

1 Introducción

El monitoreo continuo de la calidad del agua es fundamental para detectar vertidos clandestinos y fallas operacionales en sistemas de tuberías y efluentes. Tradicionalmente, este proceso depende de la toma de muestras manuales o de equipos ópticos de alto costo.

El objetivo principal de este proyecto es desarrollar y validar un sistema usando el modulo AS7265x, teniendo los siguientes objetivos:

- La conexión efectiva del módulo AS7265x a un microprocesador del tipo ESP32
- El diseño de un código capaz de captar y mostrar los datos en tiempo real.
- La construcción de un sistema capaz de validar el funcionamiento del módulo y código funcionando de forma simultánea.

2 Marco teórico

2.1 Espectroscopia de Absorción

El sistema se basa en el principio de que diferentes contaminantes absorben o dispersan la luz en longitudes de onda específicas. El sensor AS7265x emite luz blanca, UV e IR, y mide la cantidad de luz que logra rebotar o atravesar el medio acuoso a través de 18 canales independientes (desde 410 nm hasta 860 nm).

2.2 Cálculo del Índice de Alteración

Para cuantificar la contaminación, el sistema no busca un químico en específico, sino cualquier desviación respecto a un estado "ideal" (agua limpia). Esto se calcula utilizando el Error Absoluto Medio (MAE) entre la lectura actual y la referencia calibrada:

$$MAE = \frac{1}{n} \sum_{i=1}^n |Actual_i - Referencia_i| \quad (1)$$

Donde n es el número de canales (n=18).

2.3 Deriva de Línea Base (Baseline Drift)

Los sensores ópticos de bajo costo sufren calentamiento térmico en sus LEDs, lo que disminuye su eficiencia luminosa con el tiempo. Para evitar falsos positivos, se desarrolló un algoritmo que actualiza la referencia lentamente si la desviación es menor a un umbral U_{drift} , pero "congela" la referencia y dispara la alarma si el cambio es súbito.

3 Metodología y Diseño Experimental

3.1 Hardware y Electrónica

El núcleo del sistema es un módulo AS7265x conectado a un ESP32 mediante interfaz UART (Pines TX/RX a 115200 baudios). El microcontrolador actúa como un puente (passthrough) con un filtro interno para asegurar la integridad de las tramas de datos antes de enviarlas al ordenador.

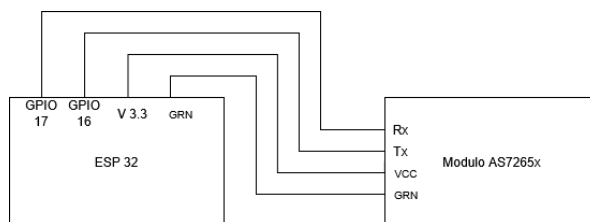


Figure 1: Gráfica conexiones

3.2 Celda de Flujo

Se construyó un circuito hidráulico cerrado. El módulo de detección consiste en una conexión de PVC equipada con una ventana de vidrio sellada. El sensor se montó en un tapón ciego exterior, garantizando el aislamiento lumínico del entorno (cámara oscura) y protegiendo la electrónica del contacto directo con el agua.

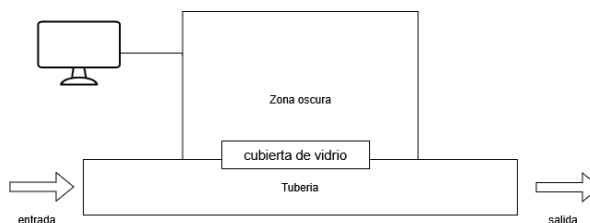


Figure 2: Gráfica montaje

3.2.1 Medidas del montaje

- PVC: 1 metro de largo.
- Flujo: $200 \frac{ml}{s}$.
- Inclinacion: 4.5.
- Cubierta de vidrio: minimo 4.5 x 5.0 cm.

3.3 Codigo ESP32

```
1 #include <HardwareSerial.h>
2
3 HardwareSerial mySerial(2);
4
5 void setup() {
6     Serial.begin(115200);
7     mySerial.begin(115200, SERIAL_8N1, 16, 17);
8     delay(2000);
9
10    mySerial.println("ATLED1=1");
11    delay(200);
12    mySerial.println("ATLED3=1");
13    delay(200);
14    mySerial.println("ATLED5=1");
15    delay(200);
16    mySerial.println("ATGAIN=3");
17 }
18
19 void loop() {
20     mySerial.println("ATCDATA");
21     delay(600);
22
23     String dataLine = "";
24     while (mySerial.available()) {
25         char c = mySerial.read();
26         if (c == '\n' || c == '\r') {
27             dataLine.trim();
28             int comaCount = 0;
29             for (int i = 0; i < dataLine.length(); i++) {
30                 if (dataLine[i] == ',') comaCount++;
31             }
32             if (comaCount >= 17) {
33                 Serial.println(dataLine);
34             }
35             dataLine = "";
36         } else {
37             dataLine += c;
38         }
39     }
40     delay(400);
41 }
```

Notar que la toma de muestras se hace cada 400 milisegundos, si es un caso de flujo rápido se puede acotar el tiempo entre muestras y en flujo lento prolongar dependiendo del caso.

3.4 Procesamiento de Datos (Software)

El script de Python aplica dos capas de filtrado:

- Filtro de Media Móvil (n=5): Para suavizar lecturas erráticas causadas por microburbujas en el flujo.
- Seguimiento Dinámico: Adaptación algorítmica de la referencia base (5%).

```
1 import serial
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4 import csv
5 from datetime import datetime
6 import os
7 import numpy as np
8 from collections import deque
9
10 # --- CONFIGURACION ---
11 PUERTO = 'COM3'
12 BAUDIOS = 115200
```

```

13 NUM_CANALES = 18
14 CSV_NAME = "datos_tuberia.csv"
15
16 # Parametros del Filtro
17 VENTANA_SUAVIZADO = 5
18 TASA_ADAPTACION = 0.05
19 UMBRAL_CONGELACION = 4.5
20 UMBRAL_ALERTA = 15.0
21
22 labels = ['410', '435', '460', '485', '510', '535', '560', '585', '610',
23           '645', '705', '760', '610b', '680', '730', '760b', '810', '860']
24
25 referencia_dinamica = np.zeros(NUM_CANALES)
26 buffer_lecturas = deque(maxlen=VENTANA_SUAVIZADO)
27 calibrado = False
28
29 # Inicializar CSV
30 if not os.path.exists(CSV_NAME):
31     with open(CSV_NAME, 'w', newline='') as f:
32         writer = csv.writer(f)
33         writer.writerow(['Timestamp'] + labels + ['Indice_Alteracion'])
34
35 try:
36     ser = serial.Serial(PUERTO, BAUDIOS, timeout=1)
37     ser.flushInput()
38     print("--- CONECTADO ---")
39     print("Deja correr el agua limpia y presiona 'R' para la calibracion inicial.")
40 except Exception as e:
41     print(f"ERROR: {e}")
42     exit()
43
44 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 9))
45 plt.subplots_adjust(hspace=0.4)
46
47 bar_actual = ax1.bar(labels, [0]*NUM_CANALES, color='#007acc', alpha=0.8, label='
48 Actual')
49 bar_ref = ax1.bar(labels, [0]*NUM_CANALES, color='gray', alpha=0.3, label='Referencia
50 Dinamica')
51 text_info = ax1.text(0.02, 0.90, 'Esperando calibracion...', transform=ax1.transAxes)
52 ax1.set_title("Espectro de Luz (Autocompensado)")
53 ax1.set_ylim(0, 1000)
54 ax1.legend()
55
56 bar_diff = ax2.bar(labels, [0]*NUM_CANALES, color='salmon')
57 ax2.set_title("Desviacion (Eliminando Deriva Termica)")
58 ax2.axhline(0, color='black')
59 text_indice = ax2.text(0.95, 0.90, 'Indice: 0.0', transform=ax2.transAxes, ha='right',
60                        fontsize=14, fontweight='bold')
61
62 def on_key(event):
63     global referencia_dinamica, calibrado
64     if event.key.lower() == 'r':
65         if len(buffer_lecturas) > 0:
66             referencia_dinamica = np.mean(buffer_lecturas, axis=0)
67             calibrado = True
68             print(">>> Calibracion Inicial: Referencia Dinamica Activada")
69
70 fig.canvas.mpl_connect('key_press_event', on_key)
71
72 def update(frame):
73     global referencia_dinamica
74
75     if ser.in_waiting > 0:
76         try:
77             line = ser.readline().decode('utf-8', errors='ignore').replace("OK", "").
78             replace("\x00", "").strip()
79             if "," in line:
80                 raw_data = [float(x.strip()) for x in line.split(',') if x.strip()]
81
82                 if len(raw_data) == NUM_CANALES:
83                     buffer_lecturas.append(raw_data)

```

```

80         datos_suavizados = np.mean(buffer_lecturas, axis=0)
81
82         if not calibrado:
83             return bar_actual, bar_ref, bar_diff
84
85         # 1. Calcular Diferencia e Indice
86         diferencia = datos_suavizados - referencia_dinamica
87         indice_alteracion = np.mean(np.abs(diferencia))
88
89         # 2. Compensacion Dinamica
90         # Si el indice es menor a UMBRAL_CONGELACION, asumimos que es
calentamiento de LEDs o microburbujas
91         if indice_alteracion < UMBRAL_CONGELACION:
92             # Acercamos la referencia hacia los datos actuales
93             referencia_dinamica = (referencia_dinamica * (1 -
TASA_ADAPTACION)) + (datos_suavizados * TASA_ADAPTACION)
94
95             # Recalcular indice despues de adaptar (tendera a bajar a 0)
96             diferencia = datos_suavizados - referencia_dinamica
97             indice_alteracion = np.mean(np.abs(diferencia))
98
99         # 3. GUARDADO en CSV
100         timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
101         with open(CSV_NAME, 'a', newline='') as f:
102             writer = csv.writer(f)
103             writer.writerow([timestamp] + list(np.around(datos_suavizados,
2)) + [round(indice_alteracion, 2)])
104
105         # 4. ACTUALIZACION GRAFICA
106         for b, v in zip(bar_actual, datos_suavizados): b.set_height(v)
107         for b, v in zip(bar_ref, referencia_dinamica): b.set_height(v)
108         for b, v in zip(bar_diff, diferencia):
109             b.set_height(v)
110             b.set_color('red' if v > 0 else 'blue')
111
112         ax1.set_ylim(0, max(np.max(datos_suavizados), np.max(
referencia_dinamica), 10) * 1.2)
113         max_d = max(np.max(np.abs(diferencia)), 5)
114         ax2.set_ylim(-max_d * 1.2, max_d * 1.2)
115
116         # Textos y Semaforo
117         color_txt = "red" if indice_alteracion > UMBRAL_ALERTA else ("
orange" if indice_alteracion > UMBRAL_CONGELACION else "green")
118         text_indice.set_text(f"Indice: {indice_alteracion:.1f}")
119         text_indice.set_color(color_txt)
120
121         except Exception:
122             pass
123
124         return bar_actual, bar_ref, bar_diff
125
126 ani = animation.FuncAnimation(fig, update, interval=100, blit=False, cache_frame_data=
False)
127 plt.show()

```

El código, a su vez, permite visualizar los cambios relativos a cada uno de los canales con la muestra actual y la prueba, permitiendo así ver fallos criticos o problemas en tiempo real.

3.5 Visualización posterior

Con el fin de poder analizar los datos obtenidos a través de la desviación de las muestras, se diseñó un código capaz de graficar está.

```

1     import pandas as pd
2     import matplotlib.pyplot as plt
3     import matplotlib.dates as mdates
4
5     # Archivo a leer
6     CSV_FILE = "Prueba.csv"
7

```

```

8
9 def analizar():
10     try:
11         print(f"Leyendo {CSV_FILE}...")
12         df = pd.read_csv(CSV_FILE)
13
14         df['Timestamp'] = pd.to_datetime(df['Timestamp'])
15
16         plt.figure(figsize=(14, 7))
17
18         plt.plot(df['Timestamp'], df['Indice_Alteracion'],
19                 color='#d62728', linewidth=2, label='Indice de Alteracion (Global)')
20
21         if '410' in df.columns:
22             max_uv = df['410'].max()
23             plt.plot(df['Timestamp'], df['410'] * (df['Indice_Alteracion'].max() /
max_uv),
24                     color='blue', alpha=0.3, label='Canal UV (Escalado)')
25
26         plt.title('Historial de Contaminacion en Tuberia', fontsize=16)
27         plt.ylabel('Nivel de Alteracion (Desviacion)', fontsize=12)
28         plt.xlabel('Tiempo', fontsize=12)
29         plt.grid(True, linestyle='--', alpha=0.7)
30
31         plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%H:%M:%S'))
32         plt.gcf().autofmt_xdate() # Rotar fechas
33
34         plt.axhline(y=15, color='orange', linestyle=':', label='Umbral de Alerta')
35
36         plt.legend()
37         plt.tight_layout()
38         plt.show()
39
40     except FileNotFoundError:
41         print("No se encontro el archivo CSV. Ejecuta primero el monitor.")
42     except Exception as e:
43         print(f"Error analizando datos: {e}")
44
45 if __name__ == "__main__":
46     analizar()

```

4 Resultados y Discusión

4.1 Estabilidad y Prueba en Blanco

Durante la fase de control con agua limpia, se observó una deriva inicial debido al calentamiento de los LEDs. Gracias al algoritmo de compensación, el Índice de Alteración se mantuvo estable por debajo de 2.0, evitando falsas alarmas.

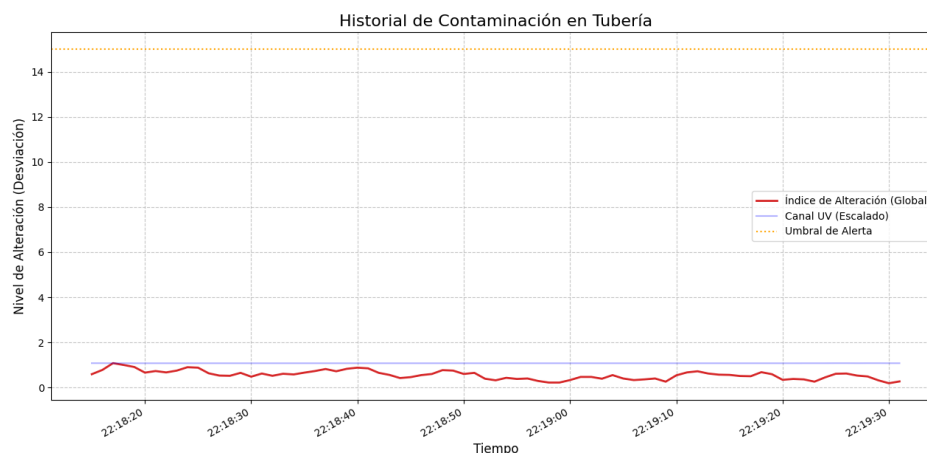


Figure 3: Gráfica control

4.2 Detección de Anomalías sin compensación dinámica

Se inyectaron 10 ml de detergente comercial, en una concentración de 10 ml de detergente en 100 ml de agua aproximadamente cada 30 segundos, en el flujo. Como se observa en la Figura 4, en la gráfica se ven las alteraciones erráticas sin una forma definida, aun siendo distinguibles los picos de contaminación, al código no adaptar la desviación se puede ver que los datos que deben ser normales poseen una desviación creciente en el tiempo.

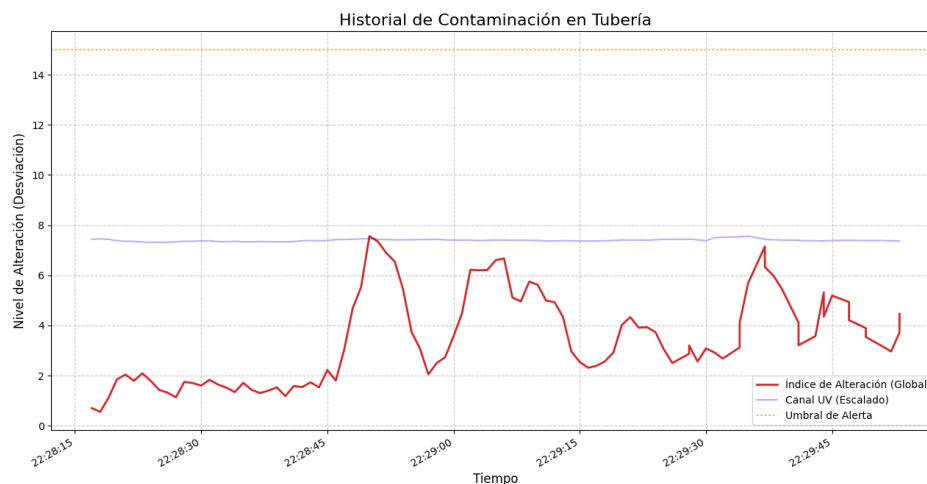


Figure 4: Gráfica prueba de detección

4.3 Detección de Anomalías con compensación dinámica

Se inyectaron 10 ml de detergente comercial, en una concentración de 10 ml de detergente en 100 ml de agua aproximadamente cada 30 segundos, en el flujo. Como se observa en la Figura 5, en la gráfica

se ven las alteraciones como picos, registrados en 4 puntos, además de apreciar el crecimiento lineal por temperatura y factores ambientales disminuidos.

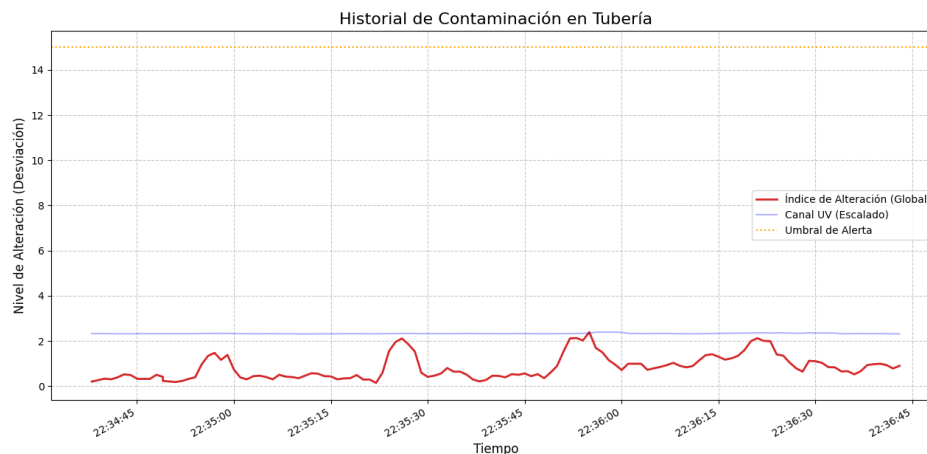


Figure 5: Gráfica prueba de detección

5 Conclusiones

Se demostró con éxito que la posibilidad de que el módulo logre detectar contaminantes en fluidos cambiantes está comprobada en un sistema de prueba. Además de que es posible analizar los datos a través de programas de software, tanto en tiempo real como diferido, con la función de realizar un análisis histórico de las alteraciones.

6 Mejoras a futuro

- Análisis del comportamiento de desviación por temperatura, para optimización del código.
- Análisis de Fourier (FFT): Aplicar transformadas de Fourier a registros de 24 horas para identificar patrones de contaminación periódicos.
- Implementación IoT: Migrar el almacenamiento de datos locales (CSV) a una base de datos en la nube vía Wi-Fi para monitoreo remoto.

7 Anexos

- <https://github.com/mavy263/AS7268X-Monitoreo-de-liquido-en-movimiento>